



Departamento de Ciencias de la Computación



Asignatura:

Modelos Discretos para la Ingeniería

Tema:

Sistema de Evaluación Académica Basado en Inferencia Lógica

Asesor:

Mgtr. Washington Eduardo Loza Herrera

Integrantes:

Chavez Oscullo Lizbeth Liliana

Galarza Alvaro Matías Miguel

Iza Caiza Melany Guadalupe

Jácome Calahorrano Micaela Victoria

NRC:30627

11/05/2026

Contenido

Introducción	4
Objetivo.....	4
Objetivo general.....	4
Objetivos específicos	5
Desarrollo.....	5
Marco Teórico.....	5
Conjuntos	5
Listas:.....	8
Pilas:.....	8
Colas:	8
Desarrollo del sistema.....	9
Implementación Patrón Arquitectónico MVC (Modelo-Vista-Controlador)	9
Nodos y Memoria Dinámica.....	9
Listas Dinámicas.....	10
Colas Dinámicas (Lógica FIFO).....	11
Pilas Dinámicas (Lógica LIFO).....	12
Lógica Proposicional	13
Inferencia Lógica	14

Teoría de Conjuntos.....	15
Conclusiones.....	16
Referencias.....	18

Introducción

El presente proyecto, denominado Sistema de Evaluación Académica Basado en Inferencia Lógica, fue desarrollado con el objetivo de integrar conceptos de programación orientada a objetos, lógica matemática y estructuras dinámicas de datos dentro de una aplicación funcional en C++. El sistema permite registrar información académica de estudiantes y, mediante el análisis de proposiciones lógicas relacionadas con promedio, asistencia y cumplimiento de tareas, determinar automáticamente su estado académico.

Para su desarrollo se empleó la arquitectura Modelo–Vista–Controlador (MVC), permitiendo separar adecuadamente la lógica del sistema, la gestión de datos y la interacción con el usuario. Además, se implementó manejo de memoria dinámica mediante nodos enlazados, utilizando listas, pilas y colas para representar distintos procesos académicos dentro del sistema.

Asimismo, el proyecto incorpora fundamentos de lógica proposicional, inferencia lógica y teoría de conjuntos, aplicando operaciones matemáticas y razonamientos automáticos para clasificar estudiantes en conjuntos de aprobados, recuperación y reprobados. De esta manera, el sistema representa una aplicación práctica de los contenidos teóricos de matemáticas discretas y estructuras de datos en un entorno computacional real.

Objetivo

Objetivo general

Aplicar modelos discretos mediante lógica proposicional, teoría de conjuntos y operaciones computacionales para clasificar, organizar y analizar el estado académico de los estudiantes en un sistema de evaluación.

Objetivos específicos

Representar las condiciones académicas de los estudiantes mediante proposiciones lógicas para determinar su estado dentro del sistema de evaluación.

Clasificar a los estudiantes en grupos según su estado académico para organizar la información de manera estructurada.

Implementar operaciones de conjuntos como unión, intersección y diferencia para analizar la relación entre los grupos de estudiantes.

Desarrollo

Marco Teórico

Conjuntos

La palabra conjunto fue introducida por Georg Cantor y, aunque no existe una única definición universal, puede entenderse como una agrupación de elementos que comparten una característica común. Un conjunto puede estar formado por números, objetos, personas o cualquier elemento que pueda identificarse claramente. De acuerdo con Epp (2012), un conjunto está completamente determinado por los elementos que están en él —no por el orden en el que se podrían utilizar o por el hecho de que algunos elementos podrían listarse más de una vez (p. 7). Por esta razón, los conjuntos permiten organizar información de manera precisa y evitar duplicaciones dentro de una misma agrupación.

En el ámbito de las matemáticas discretas y la computación, los conjuntos son importantes porque permiten clasificar datos y analizar la relación entre diferentes grupos de elementos. Su utilidad se observa cuando se necesita organizar información según características específicas, como ocurre en un sistema académico al agrupar estudiantes de

acuerdo con su estado de aprobación, recuperación o reprobación. De esta manera, los conjuntos permiten estructurar la información para facilitar su análisis posterior.

Conjunto Universal.

Un conjunto universal es aquel que contiene todos los elementos considerados dentro de un contexto determinado. A partir de este conjunto general se pueden formar distintos subconjuntos, los cuales permiten separar la información en grupos más específicos según las características que se desean analizar.

En un contexto académico, el conjunto universal puede estar formado por todos los estudiantes registrados.

Subconjuntos

Un subconjunto es aquel cuyos elementos también pertenecen a otro conjunto, es decir, todos los elementos de un subconjunto deben estar contenidos dentro de un conjunto principal. Esto permite organizar la información de forma jerárquica y separar los datos en categorías más específicas.

Según Epp (2012), cuando un conjunto no es subconjunto de otro, existe al menos un elemento del primer conjunto que no pertenece al segundo (p. 9). Por ello, la condición principal para identificar un subconjunto es que todos sus elementos formen parte del conjunto mayor.

Los subconjuntos son útiles porque permiten analizar una parte determinada de los datos sin separarla completamente del conjunto principal. Por ejemplo, dentro del conjunto universal de estudiantes, se pueden formar subconjuntos de estudiantes aprobados, estudiantes en recuperación o estudiantes reprobados. Cada uno de estos grupos conserva relación con el conjunto general, pero representa una categoría específica.

Unión de conjuntos.

La unión de conjuntos es una operación que permite formar un nuevo conjunto con los elementos que pertenecen a uno u otro conjunto, o a ambos, sin repetir elementos. En otras palabras, si se tienen dos conjuntos, la unión reúne todos los elementos que forman parte del primer conjunto, del segundo conjunto o de los dos al mismo tiempo. De acuerdo con Lipschutz (1991), la unión de dos conjuntos está formada por todos los elementos que pertenecen a un conjunto, al otro o a ambos. Esta operación permite agrupar información de diferentes conjuntos en una sola colección.

Intersección de conjuntos.

La intersección de conjuntos es una operación que permite obtener un nuevo conjunto formado únicamente por los elementos comunes entre dos o más conjuntos. Es decir, está compuesta por aquellos elementos que pertenecen simultáneamente a un conjunto y también a los demás. De acuerdo con Lipschutz (1991), la intersección corresponde al conjunto de objetos que se encuentran tanto en un conjunto como en otro, o en varios conjuntos a la vez. Esta operación permite identificar coincidencias entre diferentes agrupaciones y analizar qué elementos comparten entre sí.

Diferencia de conjuntos.

La diferencia de conjuntos es una operación que permite formar un nuevo conjunto con los elementos que pertenecen a un conjunto, pero no pertenecen a otro. Es decir, si se tienen dos conjuntos, esta operación permite identificar aquellos elementos que son propios del primer conjunto y que no se encuentran en el segundo.

De acuerdo con Lipschutz (1991), “la diferencia de los conjuntos A y B es el conjunto de los elementos que pertenecen al conjunto A pero no al conjunto B, y se denota como $A - B$ ”.

B". Esta operación es útil para comparar conjuntos y determinar qué elementos son exclusivos de una agrupación determinada.

Listas:

Según (*Lista De C++ (Con Ejemplos)*, n.d.) , son aquellos contenedores de la biblioteca STL que almacenan elementos de forma no contigua en la memoria, esto enlazándolos mediante punteros, permiten inserciones y eliminaciones rápidas en cualquier posición pero no ofrecen un acceso aleatorio.

Pilas:

Según (*W3Schools.com*, n.d.) es una estructura de datos lineales, mismas que se rigen por el principio LIFO (Last in, First out); es decir, se puede expresar como que el primer elemento ingresado será el último en salir durante un proceso, mientras que el último en ingresar será el primero en salir.

Se diferencian de los vectores ya que los elementos de las pilas no se accede mediante índices, dado que se añaden y eliminan desde la parte superior.

Colas:

Como lo explica (*W3Schools.com*, n.d.), estas estructuras emplean el orden FIFO (First in, first out), es decir, el primer elemento en ingresar es el primer elemento en salir; muchos textos lo ejemplifican con una fila en el supermercado.

Es así como esta forma de organizar los elementos, solo permite acceder al primero o último elemento con los que se esté tratando; además de emplear la librería <queue>.

Desarrollo del sistema

Implementación Patrón Arquitectónico MVC (Modelo-Vista-Controlador)

El proyecto fue diseñado separando estrictamente las responsabilidades mediante el patrón MVC. Esto garantiza un código escalable, mantenible y de bajo acoplamiento.

Modelo: Encapsula la lógica de negocio, las estructuras de datos (Pilas, Colas, Listas) y las reglas matemáticas/lógicas (Estudiante, ConjuntoEstudiantes, MotorInferencia).

Vista: Gestiona exclusivamente las entradas del usuario (cin) y las salidas a consola (cout) sin realizar procesos lógicos (VistaGestionEstudiantes).

Controlador: Actúa como el puente; solicita datos a la Vista, los procesa mediante el Modelo y envía las respuestas procesadas de vuelta a la Vista (ControladorGestionEstudiantes).

Nodos y Memoria Dinámica

A diferencia de sistemas estáticos que usan arreglos de tamaño fijo ([]) o librerías abstractas (std::vector), este sistema gestiona su propia memoria en tiempo de ejecución (Heap) mediante apuntadores. Un Nodo es la unidad estructural básica que contiene un dato y apunta al siguiente, formando una cadena en la memoria que crece o se reduce según se necesite, previniendo el desperdicio de memoria.

Código (nodo.h):

```
struct Nodo {
```

```
    Estudiante dato;
```

```
    Nodo *siguiente; // Apuntador al siguiente bloque de memoria
```

```

Nodo(Estudiante e) {

    dato = e;

    siguiente = nullptr;

}

};

```

Código (EstructurasDinamicas.cpp):

```

// Creación dinámica de memoria (new)

Nodo* nuevo = new Nodo(e);

// Liberación explícita de memoria (delete)

Nodo* borrar = tope;

tope = tope->siguiente;

delete borrar;

```

Listas Dinámicas

Se implementó una Lista Simplemente Enlazada (ListaEstudiantes) para crear repositorios de almacenamiento general de estudiantes. Esta lista permite iterar, insertar al final y eliminar nodos intermedios de forma eficiente ajustando los apuntadores.

Código (Estructuras Dinámicas.cpp):

```

void ListaEstudiantes::agregar(const Estudiante& e) {

    Nodo* nuevo = new Nodo(e);

```

```

if (cabeza == nullptr) {

    cabeza = nuevo;

} else {

    Nodo* actual = cabeza;

    while (actual->siguiente != nullptr) { // Iteración de la lista

        actual = actual->siguiente;

    }

    actual->siguiente = nuevo;

}

}

```

Colas Dinámicas (Lógica FIFO)

Las colas funcionan bajo el principio FIFO (First In, First Out - Primero en entrar, primero en salir). Fueron utilizadas dentro del proceso de evaluación masiva del motor de inferencia. La lógica estipula que los estudiantes deben ser evaluados en el mismo orden en el que fueron ingresados al sistema.

Código (ControladorGestionEstudiantes.cpp):

```

// Encolamos (FIFO) a todos los estudiantes pendientes de evaluación

ColaEstudiantes colaEvaluacion;

Nodo* actual = estudiantes;

```

```

while (actual != nullptr) {

    colaEvaluacion.encolar(actual->dato); // Push

    actual = actual->siguiente;

}

// Desencolamos en orden de llegada para procesarlos lógicamente

while (!colaEvaluacion.vacia()) {

    Estudiante e = colaEvaluacion.verFrente();

    colaEvaluacion.desencolar(); // Pop

    // ... evaluación

}

```

Pilas Dinámicas (Lógica LIFO)

Las pilas operan bajo el principio LIFO (Last In, First Out - Último en entrar, primero en salir). En el sistema, la pila es utilizada explícitamente en el proceso de Recuperación. Su justificación en la vida real recae en que, cuando un profesor recoge los exámenes de recuperación, los va apilando en su escritorio. Al momento de calificar, retira y procesa primero el examen que quedó en el "Tope" de la pila (el último estudiante en entregarlo).

Código (ControladorGestionEstudiantes.cpp):

```

// Pila para calificar exámenes de recuperación (LIFO)

PilaEstudiantes pilaExamenes;

// Apilamos a los estudiantes que rinden recuperación

```

```

while (actualB != nullptr) {

    pilaExamenes.apilar(actualB->dato);

    actualB = actualB->siguiente;

}

// Desapilamos (LIFO) y procesamos cada examen

while (!pilaExamenes.vacia()) {

    Estudiante e = pilaExamenes.desapilar();

    // ... estudiante aprueba y su nota es modificada.

}

```

Lógica Proposicional

Las bases del razonamiento del sistema parten de declarar proposiciones lógicas simples (afirmaciones que solo pueden ser Verdaderas o Falsas), extrayendo sus valores de verdad a partir de atributos cuantitativos del estudiante.

Código (Estudiante.cpp):

```

// Proposición P: El estudiante tiene un promedio final >= 7

bool Estudiante::evaluarP() const {

    return promedio >= 7.0;

}

// Proposición Q: La asistencia del estudiante es >= 80%

```

```

bool Estudiante::evaluarQ() const {

    return asistencia >= 80.0;

}

// Proposición R: Entregó todas sus actividades obligatorias

bool Estudiante::evaluarR() const {

    return tareasCompletas;

}

```

Inferencia Lógica

Corresponde al proceso de derivar conclusiones lógicas válidas uniendo las premisas (P, Q y R) mediante operadores lógicos booleanos: Conjunción (&& AND), Disyunción (|| OR) y Negación (! NOT). Con esto el sistema clasifica de forma determinista el futuro académico del estudiante sin necesidad de condicionales anidados complejos.

Código (MotorInferencia.cpp):

```

EstadoAcademico MotorInferencia::deducirEstado(const Estudiante &estudiante) {

    bool p = estudiante.evaluarP();

    bool q = estudiante.evaluarQ();

    bool r = estudiante.evaluarR();

    // Regla 1: Aprueba directamente si cumple P, Q y R ( $P \wedge Q \wedge R$ )

    if (p && q && r) {

```

```

        return APROBADO;

    }

    // Regla 2: Va a recuperación si falla el promedio pero tiene asistencia ( $\sim P \wedge Q$ ),

    // o si cumple promedio y asistencia pero falló en tareas ( $P \wedge Q \wedge \sim R$ ).

    else if ((!p && q) || (p && q && !r)) {

        return RECUPERACION;

    }

    // Regla 3: En cualquier otro caso (como reprobar asistencia  $\sim Q$ ), reprueba.

    else {

        return REPROBADO;

    }

}

```

Teoría de Conjuntos

Tras inferir el estado de cada estudiante, estos son clasificados dentro de Conjuntos Matemáticos discretos (A, B y C). La clase ConjuntoEstudiantes no acepta elementos duplicados, y permite ejecutar operaciones fundamentales de la teoría matemática sobre ellos manipulando dinámicamente los nodos de las listas que albergan.

Código (ConjuntoEstudiantes.cpp):

```

// Operación Diferencia (A - C)

```

```

ConjuntoEstudiantes ConjuntoEstudiantes::diferencia(const ConjuntoEstudiantes&
otro, string nuevoNombre) const {

    ConjuntoEstudiantes resultado(nuevoNombre);

    Nodo* actual = elementos.getCabeza();

    while (actual != nullptr) {

        // Si el estudiante está en este conjunto PERO NO pertenece al otro:

        if (!otro.pertenece(actual->dato.getNombre())) {

            resultado.agregar(actual->dato);

        }

        actual = actual->siguiente;

    }

    return resultado;

}

```

Conclusiones

El desarrollo del Sistema de Evaluación Académica Basado en Inferencia Lógica permitió integrar de manera práctica diversos conceptos fundamentales de programación, lógica matemática y estructuras dinámicas de datos dentro de una aplicación funcional en C++. A través de este proyecto se evidenció cómo la lógica proposicional y las reglas de

inferencia pueden utilizarse para automatizar procesos de decisión académica de forma objetiva y consistente.

La implementación del patrón arquitectónico MVC contribuyó a mantener una adecuada separación de responsabilidades entre la interfaz, la lógica de control y el procesamiento de datos, mejorando la organización, mantenibilidad y escalabilidad del sistema. Del mismo modo, el uso exclusivo de memoria dinámica mediante nodos fortaleció el manejo de punteros, asignación dinámica y liberación explícita de memoria, evitando dependencias de estructuras abstractas prediseñadas.

La incorporación de listas enlazadas, pilas y colas permitió aplicar estructuras dinámicas reales a escenarios académicos concretos. Las listas facilitaron el almacenamiento flexible de estudiantes, las colas modelaron procesos de evaluación en orden de llegada y las pilas representaron adecuadamente la revisión de exámenes de recuperación bajo comportamiento LIFO. Asimismo, la teoría de conjuntos permitió clasificar y relacionar estudiantes mediante operaciones matemáticas como unión, intersección y diferencia.

En el ámbito lógico, el sistema logró representar proposiciones booleanas y aplicar operadores de conjunción, disyunción y negación para inferir automáticamente el estado académico de cada estudiante. Esto demuestra la capacidad de trasladar principios teóricos de matemáticas discretas hacia soluciones computacionales aplicadas.

Referencias

- Lipschutz, S. (1991). Teoría y problemas de teoría de conjuntos y temas afines. McGraw-Hill.
- Epp, S. S. (2012). Matemáticas discretas con aplicaciones. Cengage Learning.
- Lista de C++ (con ejemplos)*. (n.d.). <https://www.programiz.com/cpp-programming/list>
- W3Schools.com. (n.d.). https://www.w3schools.com/cpp/cpp_stacks.asp
- W3Schools.com. (n.d.). https://www.w3schools.com/cpp/cpp_queues.asp